Explaining quantum phase estimation using data structures

(Dated: February 27, 2026)

Abstract

The quantum phase estimation algorithm is a fundamental quantum computing technique used in physical simulation and linear algebraic tasks. It is typically taught by stating the quantum circuit for quantum phase estimation and then proving its correctness by algebraic symbol manipulation. This approach rigorously establishes the correctness of the algorithm, but it does not provide a high-level algorithmic perspective on what the algorithm is doing. This paper explains the phase estimation algorithm using high-level concepts, by showing that quantum phase estimation employs elementary polynomial data structures from classical computer science. Once the polynomial data structure concepts have been taught to students, the explanation of the quantum phase estimation algorithm can be stated in just a few lines. As quantum computing continues to grow in popularity and is taught in more diverse contexts, we hope this alternative approach to explaining phase estimation will be useful to students and instructors.

I. INTRODUCTION

Quantum computers promise new algorithms for solving problems that are intractable on classical computers[1]. Their power comes from quantum bits (qubits), which, unlike classical bits, can exploit superposition, interference, and entanglement to store and process information more efficiently. Among the most important algorithms that harness this power is quantum phase estimation[2]. It is a subroutine in seminal algorithms such as in the simulation of physical systems[3] and in Shor's factoring algorithm[4], and it is continues to be employed in novel quantum algorithms[5,6].

Existing presentations of quantum phase estimation state the quantum circuit for the algorithm and present an analysis of the correctness of the algorithm by means of algebraic symbol manipulation[1,7]. While this approach is rigorous, we find the symbolic reasoning to be cumbersome. Moreover, we think that it obscures the algorithmic ideas of quantum phase estimation, since students are not explained how they may have discovered the algorithm by themselves, or how it may be similar to other algorithms in classical or quantum computer science. This could make it hard for students to remember how quantum phase estimation works, or to intuitively reason about variants of the algorithm without performing laborious derivations. It would be better if the rigorous algebraic details were presented only after some high-level ideas that capture how the algorithm works.

The usefulness of this paper is in providing such an alternative explanation for quantum phase estimation that is described in terms of algorithmic ideas that are well-known in classical computer science. We show how data structures for polynomials and a divide-and-conquer framing explain the quantum phase estimation algorithm in a simple way. Note that data structures are an essential concept in the study of classical algorithms. It is common for the crux of an algorithm to be a clever data structure that makes the storage and processing of information in that computational task more efficient. Although quantum phase estimation has not been described in this way before, we show how such a data structure approach simplifies its presentation and points to the source of the quantum advantage.

An ever-larger community of scientists will need at least a working understanding of phase estimation because quantum computing is a growing scientific area and industry in its own right, and it is expected that it will be applied in various branches of science. With the

rapid growth of university courses and programs in quantum computing, many instructors across physics and other departments will find themselves teaching or applying it. We hope our conceptual explanation will be helpful to these instructors. Instructors who use our explanation in their courses, may also decide to include the usual algebraic details if this is appropriate, whether as further course material or incorporated into homework exercises.

II. DATA STRUCTURES FOR PHASE ESTIMATION

This section introduces the concepts that feature in our explanation of quantum phase estimation. We first review classical data structures for representing polynomials, then show how these motivate the Discrete and Fast Fourier transforms, and finally explain the quantum analogues of these ideas. Table I condenses these concepts and how they are related.

	Coefficient Representation	**Point-value Representation**
Classical	$[\, a_0 \quad a_1 \quad \dots \quad a_{N-1} \,]$	$[\, p(1) \quad p(\omega_N) \quad \dots \quad p(\omega_N^{N-1}) \,]$
Quantum (Amplitude encoding)	$\frac{1}{\|\|a\|\|} \sum_{j=0}^{N-1} a_j \, \|j\rangle$	$\frac{1}{\sqrt{N}\|\|a\|\|} \sum_{j=0}^{N-1} p(\omega_N^j) \, \|j\rangle$

TABLE I: Data structures for storing the polynomial $p(x) = a_0 + ... + a_{N-1}x^{N-1}$. The two classical data structures (top row) are stored as a list of values or a vector. The two quantum data structures (bottom row) are quantum states known as the *amplitude encodings* of these vectors. We can convert from the coefficient representation to the point-value representation by applying the Discrete Fourier Transform (DFT) in the classical case, and the Quantum Fourier Transform (QFT) in the quantum case. To convert in the other way, the inverse-DFT or inverse-QFT may be used.

A. Polynomial data structures

The material in this subsection can be motivated by asking students to consider how polynomials might be represented in a classical computer. The straightforward approach, the **coefficient representation** of a polynomial[8] involves storing a vector (or list) of all its

coefficients. For example the polynomial $p(x) = a_{N-1}x^{N-1} + ... + a_0$ is stored as the vector $a = [a_0...a_{N-1}]$.

There are other data structures for polynomials. Consider the **point-value representation** of a polynomial[8], which uses the fact that a polynomial, p, with degree less than N is uniquely determined by a set of N distinct points $\{(x_0, p(x_0)), ..., (x_{N-1}, p(x_{N-1}))\}$. For example, storing two points represents the unique line that passes through them. For a polynomial of degree less than N, we can evaluate it at N different points and store the identities of those points. Note that we are free to choose any set of N distinct input values for this purpose, and regardless of our choice these points uniquely identity the polynomial. It is convenient to use the same set of input values for all the different polynomials we are storing. If we use the inputs $\{x_0, ..., x_{N-1}\}$ to create point-value representations, then any polynomial, p of degree up to N can be stored as the vector $[p(x_0)...p(x_{N-1})]$. This is explained with examples in Figure 1.

The point-value representation is significant in classical algorithms because multiplying two polynomials that are represented in this way is efficient. For two polynomials p and q, their product $r = p \cdot q$ can be obtained simply by multiplying their stored values coordinate-wise:

$$[r(x_0), \ldots, r(x_{N-1})] = [p(x_0)q(x_0), \ldots, p(x_{N-1})q(x_{N-1})]. \tag{1}$$

This procedure works as long as the polynomials p and q each have degree less than $N/2$ so that the resulting product r still has degree less than N (and can thus be uniquely determined by N points). Multiplication of polynomials in the point-value representation only requires $\mathcal{O}(N)$ multiplications of numbers. In contrast, multiplication of polynomials in the coefficient representation is done by long multiplication[9] which requires $\mathcal{O}(N^2)$ additions and multiplications of numbers to combine and sum cross-terms.

The coefficient representation is the more common and natural choice, so one would hope to store polynomials in the coefficient representation but use a method to convert between the two representations in order to take advantage of the faster multiplication algorithm in the point-value representation. However this idea only works if we convert between representations efficiently. The naive approach to converting from coefficients to point values requires multiplying by a Vandermonde matrix[10]. For example, given $p(x) = a_0 + a_1x + \cdots + a_{N-1}x^{N-1}$ and fixed evaluation points $\{x_0, \ldots, x_{N-1}\}$, the point value representation

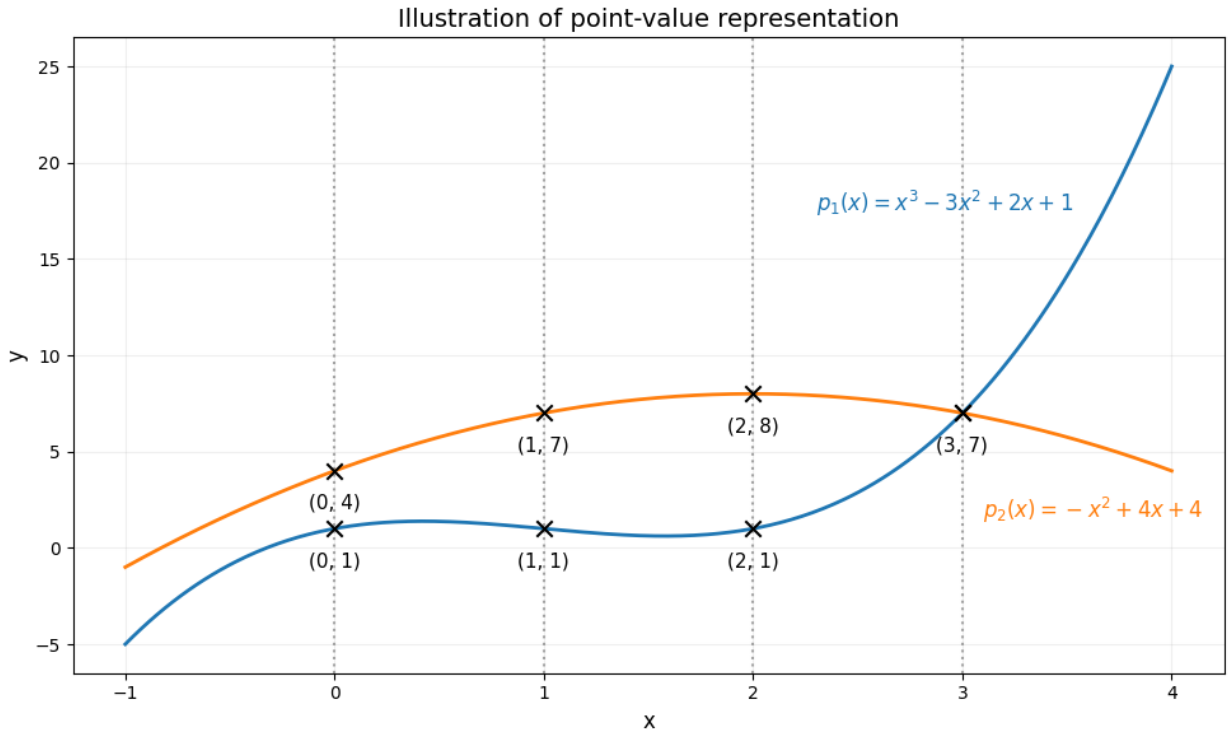

FIG. 1: There is only one polynomial of degree less than four that passes through the four points $\{(0,1),(1,1),(2,1),(3,7)\}$, and it is $x^3 - 3x^2 + 2x + 1$. Similarly the polynomial $-x^2 + 4x + 4$ is the only polynomial of degree less than four that passes through the four points $\{(0,4),(1,7),(2,8),(3,7)\}$. Thus we may represent the polynomials by these sets of points. If we use inputs $x = 0,1,2,3$ to create point-value representations, then we may simply store p_1 as $[1,1,1,7]$, and p_2 as $[4,7,8,7]$.

is obtained by:

$$\begin{bmatrix} p(x_0) \\ p(x_1) \\ \vdots \\ p(x_{N-1}) \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{N-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N-1} & x_{N-1}^2 & \cdots & x_{N-1}^{N-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{N-1} \end{bmatrix}. \tag{2}$$

In general, multiplying a matrix onto a vector requires $\mathcal{O}(N^2)$ elementary operations, and thus the naive approach to converting between the coefficient and point-value representations requires $\mathcal{O}(N^2)$ time. However directly performing the multiplication in the coefficient representation also takes $O(N^2)$ time, and thus a slow $O(N^2)$ conversion can offer us no shortcuts to multiplying polynomials.

However it is possible to convert representations in $O(N \log N)$ time if we choose a special set of evaluation points to construct the point-value representation. Specifically, if we choose the N-th roots of unity (i.e. powers of $\omega_N = e^{i\frac{2\pi}{N}}$) then we get the following Vandermonde matrix:

$$F_N = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_N & \omega_N^2 & \cdots & \omega_N^{N-1} \\ 1 & \omega_N^2 & \omega_N^{2\cdot 2} & \cdots & \omega_N^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_N^{N-1} & \omega_N^{2(N-1)} & \cdots & \omega_N^{(N-1)(N-1)} \end{bmatrix}. \tag{3}$$

This matrix (F_N) is called the discrete Fourier transform (DFT) matrix, and it is an important concept in signal processing, functional analysis and abstract algebra. Applying it to the coefficient vector $[a_0, \ldots, a_{N-1}]$ computes the values $p(1), p(\omega_N), \ldots, p(\omega_N^{N-1})$. The Fast Fourier Transform (FFT) is a divide-and-conquer method to multiply F_N to a vector in only $O(N \log N)$ time, making the conversion between coefficient and point-value representations more efficient.

The key idea is to split the N coefficients $(a_0, \ldots, a_{N-1})$ into their even- and odd-indexed parts, forming two smaller polynomials of size $N/2$. These smaller polynomials are only evaluated at even powers of ω_N and so only need to be evaluated at half as many inputs. Define $p_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + ...$ and $p_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + ...$, so that $p(x) \equiv p_{\text{even}}(x^2) + x \cdot p_{\text{odd}}(x^2)$.

Notice that:

$$\omega_N^{2(i+N/2)} = \omega_N^{2i}\, \omega_N^N = \omega_N^{2i}, \tag{4}$$

which means,

$$p_{\text{even}}((\omega_N^{i+N/2})^2) = p_{\text{even}}((\omega_N^i)^2), \text{and} \tag{5}$$

$$p_{\text{odd}}((\omega_N^{i+N/2})^2) = p_{\text{odd}}((\omega_N^i)^2). \tag{6}$$

Thus we only need to evaluate $p_{\text{even}}(x^2)$ (or $p_{\text{odd}}(x^2)$) at half the inputs ω_N^i, namely when $i = 0, 1, ..., N/2 - 1$, as the remaining values will be repetitions. We can then combine the evaluations using simple additions and multiplications as:

$$p(\omega_N^k) = p_{\text{even}}(\omega_N^{2k}) + \omega_N^k\, p_{\text{odd}}(\omega_N^{2k}), \qquad k = 0, \ldots, N-1. \tag{7}$$

The time complexity of this approach is thus given by the recursive function $T(N) = 2 \cdot T(N/2) + \mathcal{O}(N)$, as each problem of evaluating the polynomial is divided into two polyno-

mial evaluation problems of half the size, and a linear time combination step. Solving this recursion shows that the Fast Fourier Transform reduces the time complexity of converting polynomial representations from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$.

B. Quantum data structures

The material in this subsection can be motivated as the quantum analogues of the classical polynomial data structures from the previous subsection.

Quantum computers are capable of performing some computations on large vectors efficiently by encoding the data into the amplitudes of a quantum state. Since an n qubit quantum state is described using a vector of 2^n amplitudes, computations on vectors encoded as quantum states are capable of achieving large advantages in space and time complexity. The standard encoding, known as the **amplitude encoding** of a vector, is the quantum state obtained by normalising the vector and putting the coefficients into amplitudes of a quantum state, for example as: $v = [v_0 ... v_{N-1}] \rightarrow \frac{1}{\|v\|} \sum_{i=0}^{N-1} v_i \left|i\right\rangle$. Our explanation of QPE features the amplitude encodings of the coefficient and point-value representations of polynomials.

The primary way that quantum computers manipulate amplitude encoded vectors is by unitary operations. When normalised by a $\frac{1}{\sqrt{N}}$ factor, the Discrete Fourier Transform is a unitary matrix. Just as the Discrete Fourier Transform converts the coefficient representation to the point-value representation, its quantum circuit version converts the *amplitude encoding* of the coefficient representation to the *amplitude encoding* of the point-value representation. This circuit, and the transformation it implements, are both called the Quantum Fourier Transform (QFT). The circuit can be implemented efficiently by using a divide-and-conquer approach just like the Fast Fourier Transform[11]. The action of QFT on an N-dimensional computational basis state can be recursively defined in terms of a QFT on an $N/2$-dimensional computational basis state as:

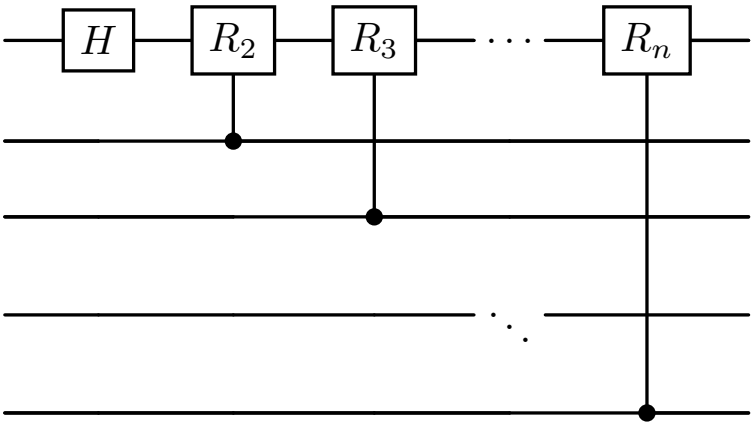

FIG. 2: The quantum circuit that uses the input computational basis state $|j\rangle$ to prepare the first qubit in the state $\frac{1}{\sqrt{2}}\begin{pmatrix}1\\ \omega_N^j\end{pmatrix}$. In this notation, $R_k = |0\rangle\langle 0| + \omega_{2^k}|1\rangle\langle 1|$. The remaining qubits are left in the state $|j \mod N/2\rangle$, which is the bitstring for j with its most significant bit removed.

$$QFT_N |j\rangle_{\text{n qubits}} = \frac{1}{\sqrt{N}} \begin{pmatrix} 1 \\ \omega_N^j \\ \vdots \\ \omega_N^{j(N-1)} \end{pmatrix} = \frac{1}{\sqrt{N/2}} \begin{pmatrix} 1 \\ \omega_N^{2j} \\ \vdots \\ \omega_N^{j(N-2)} \end{pmatrix} \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ \omega_N^j \end{pmatrix}$$

$$= QFT_{N/2} |j \mod (N/2)\rangle_{\text{n-1 qubits}} \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ \omega_N^j \end{pmatrix}, \quad (8)$$

where $n = \log_2 N$. The circuit to prepare a qubit in the state $\frac{1}{\sqrt{2}}\begin{pmatrix}1\\ \omega_N^j\end{pmatrix}$ starting from $|j\rangle$ is shown in Figure 2. It relies on the base-2 representation of $|j\rangle$ as $|b_{n-1}\dots b_0\rangle$, and the product $\omega_N^j = \omega_{2^1}^{b_{n-1}} \dots \omega_{2^{n-1}}^{b_1} \omega_{2^n}^{b_0}$.

Figure 3 shows how this divide-and-conquer approach gives a recursive definition of the QFT circuit, and Figure 4 shows an explicit definition of the QFT circuit in terms of elementary logic gates, when this recursion is solved. The QPE algorithm uses the inverse-QFT circuit. This can be obtained by simply reversing the QFT circuit and inverting every gate.

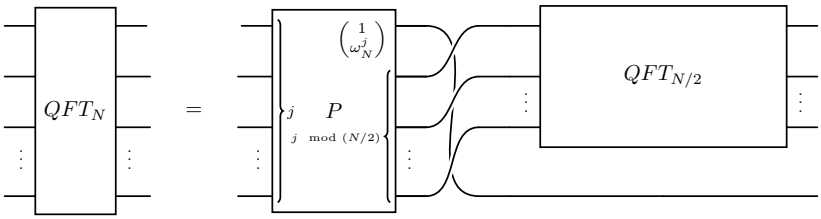

FIG. 3: Recursive definition of the quantum Fourier transform circuit. The circuit for the operation P is shown in Figure 2.

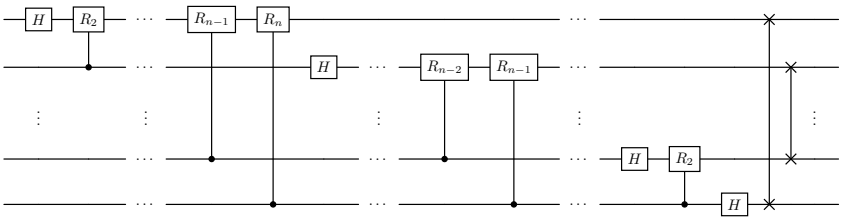

FIG. 4: Explicit quantum circuit for the quantum Fourier transform, which can be obtained by solving the recursion in Figure 3.

III. QUANTUM PHASE ESTIMATION

A. Problem

This subsection motivates the Quantum Phase Estimation problem as a natural quantum analogue of an eigenvalue problem.

Classical algorithms take $\mathcal{O}(n^3)$ time to find[12] all the eigenvalues of a given matrix. On the other hand, if an eigenvector v of a matrix A is known, the *corresponding eigenvalue* can be found in $\mathcal{O}(n^2)$ time by computing $\frac{v^T A v}{v^T v}$. Since quantum computers natively operate large matrices and vectors, the use of quantum computers for linear algebraic tasks has been an active field of research.

Quantum phase estimation is the quantum analogue of the task of computing the *corresponding eigenvalue* when given a matrix and an eigenvector. A key difference between the QPE problem and its classical counterpart is how the data is provided. In QPE, the eigenvector is given as a quantum state in which the vector is amplitude encoded, and the matrix is given as a quantum circuit of which we are given a description. Note that this

complicates the comparison of the classical and quantum algorithms, as they only solve analogous but not exactly the same problems. One way to put them on an equal footing is to view QPE as an extra tool for the classical corresponding eigenvalue problem, that can be used if the input data meets the extra requirements of the quantum algorithm. This can occur when the matrix and eigenvector are too large to operate on directly but still have some structure that makes it easy to work with them on a quantum computer. Notably, this occurs in the case of Hamiltonian simulation, where the unitary operations corresponding to the time evolutions of many interesting Hamiltonians can be implemented as efficient quantum circuits[13].

We may state the QPE problem formally as follows:

Problem. *Let U denote a unitary matrix, and let $|\psi\rangle \in \mathbb{C}^N$ be an eigenvector of U with corresponding eigenvalue $\omega_N^k = e^{i2\pi k/N}$ for some $k \in \{0, 1, ..., N-1\}$, the objective is to determine k given:*

1. *$|\psi\rangle$ as one copy of the quantum state, and*

2. *access to U via black-box calls to a qudit-controlled-U^d gate.*

We define the qudit-controlled-U^d gate, as the gate that acts in the following manner: $(C_d U^d)\,|j\rangle\,|\phi\rangle = |j\rangle\,U^j\,|\phi\rangle$. This is easily motivated as a generalisation of the standard controlled-U gate, which either applies the U gate 0 or 1 times, depending on the value of the controlling qudit. The qudit-controlled-U^d gate applies the U gate j times, if the value of the controlling qudit is j. This allows us to apply U different numbers of times in a controlled way.

B. Algorithm

This subsection shows how the high-level concepts above can be used to explain the QPE algorithm in just a few lines.

The Quantum Phase Estimation algorithm is a quantum circuit (Figure 5) that uses the qudit-controlled-U^d gate and the quantum state $|\psi\rangle$ to determine the eigenvalue of $|\psi\rangle$ with respect to U. This circuit has two parts. The first part **prepares an amplitude encoding**

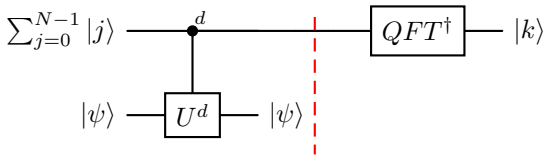

FIG. 5: High level circuit explaining the Quantum Phase Estimation Algorithm. The algorithm has two steps, separated by the dashed line. Step 1 prepares $\frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} (\omega_N^k)^j |j\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} (\omega_N^j)^k |j\rangle$, which is the amplitude encoding of the point-value representation of the polynomial x^k. Step 2 converts to the amplitude encoding of the coefficient representation, yielding k.

of the point-value representation of the polynomial $\mathbf{x^k}$. This can be shown as follows:

$$\begin{aligned} & C_d U^d [\tfrac{1}{\sqrt{N}} \textstyle\sum_{j=0}^{N-1} |j\rangle |\psi\rangle] \\ &= \tfrac{1}{\sqrt{N}} \textstyle\sum_{j=0}^{N-1} (\omega_N^k)^j |j\rangle |\psi\rangle \quad (\text{Since } U|\psi\rangle = \omega_N^k |\psi\rangle) \\ &= \tfrac{1}{\sqrt{N}} \textstyle\sum_{j=0}^{N-1} (\omega_N^j)^k |j\rangle |\psi\rangle \,. \end{aligned} \tag{9}$$

Notice that the resultant state has amplitudes equal to taking each root of unity to the k-th power, and is thus an encoding of the polynomial x^k.

The second part **converts to the amplitude encoding of the coefficient representation of the polynomial** using the inverse-QFT, giving us the outcome $|k\rangle$ since the encoded polynomial was x^k.

The running time of this algorithm depends heavily on the circuit complexity of implementing the qudit-controlled-U^d operation. When this can be done efficiently in time that is polylogarithmic in N, the QPE algorithm is capable of achieving exponential speedups over classical algorithms that use time polynomial in N to operate on the explicit matrix, U.

IV. CONCLUSION

This paper first explained the following high-level computer science concepts: data structures for polynomials, the discrete and fast Fourier transforms, amplitude encodings and the quantum Fourier transform. The two main themes in these ideas were: 1) the study of data structures, which concerns different ways in which data can be stored and their relative strengths and weaknesses; and 2) divide-and-conquer approaches to algorithm design.

Our paper is then able to explain the quantum phase estimation algorithm very quickly, as the high level concepts make it intuitive to understand what the algorithm is doing. We also showed how this intuitive understanding of what the algorithm is doing makes it easy to derive the quantum circuit to be used. Using our approach saves instructors and students from a complex proof relying on algebraic symbol manipulation that is used in most presentations of QPE. In order to receive these benefits however our explanation relies on teaching some material on data structures which is not typical of quantum computing courses. We believe this material on data structures is easy to grasp as it mostly serves to give names and interpretations that help the student understand and remember the details of the mathematics of QPE.

We believe this teaching approach leaves students with a number of easy-to-remember and reusable computer science ideas which they may combine to develop quantum algorithms themselves. In particular, data structures are emerging as a new perspective to describe quantum algorithms. A data structure perspective has similarly been used to explain the Deutsch-Jozsa algorithm[14] in an intuitive way. Data structure approaches have also been suggested for quantum algorithmic primitives[15,16] that can be used by programmers without the need for understanding underlying quantum information. We hope further investigations will recast more quantum algorithms in terms of data structures, and use the perspective to develop new algorithms.

Author Declarations

The authors have no conflicts to disclose.

---

[1] M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*. Cambridge university press, 2010.

[2] A. Y. Kitaev, "Quantum measurements and the abelian stabilizer problem," *arXiv preprint quant-ph/9511026*, 1995.

[3] D. S. Abrams and S. Lloyd, "Quantum algorithm providing exponential speed increase for finding eigenvalues and eigenvectors," *Physical Review Letters*, vol. 83, no. 24, p. 5162, 1999.

[4] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings 35th annual symposium on foundations of computer science*, pp. 124–134, Ieee, 1994.

[5] A. Schmidhuber, R. O'Donnell, R. Kothari, and R. Babbush, "Quartic quantum speedups for planted inference," *Phys. Rev. X*, vol. 15, p. 021077, Jun 2025.

[6] D. W. Berry, Y. Su, C. Gyurik, R. King, J. Basso, A. D. T. Barba, A. Rajput, N. Wiebe, V. Dunjko, and R. Babbush, "Analyzing prospects for quantum advantage in topological data analysis," *PRX Quantum*, vol. 5, no. 1, p. 010319, 2024.

[7] T. G. Wong, *Introduction to classical and quantum computing*. Rooted Grove Omaha, NE, USA, 2022.

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Polynomials and the FFT*, ch. 30, pp. 898–925. MIT Press, 3rd ed., 2009.

[9] L. West and N. Bellevue, "An introduction to various multiplication strategies," *Washington. USA. Merrill Prentice Hall*, 2011.

[10] A. Klinger, "The vandermonde matrix," *The American Mathematical Monthly*, vol. 74, no. 5, pp. 571–574, 1967.

[11] G. Paradisi and H. Randriam, "A presentation of the quantum fourier transform from a recursive viewpoint," *arXiv preprint quant-ph/0411069*, 2004.

[12] L. N. Trefethen and D. Bau, *Numerical linear algebra*. SIAM, 2022.

[13] A. Aspuru-Guzik, A. D. Dutoi, P. J. Love, and M. Head-Gordon, "Simulated quantum computation of molecular energies," *Science*, vol. 309, no. 5741, pp. 1704–1707, 2005.

[14] S. Shukla, "A good first impression for quantum algorithms," in *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 03, pp. 129–133, 2025.

[15] J. Kallaugher, O. Parekh, and N. Voronova, "How to design a quantum streaming algorithm without knowing anything about quantum computing," in *2025 Symposium on Simplicity in Algorithms (SOSA)*, pp. 9–45, SIAM, 2025.

[16] L. Zhao, C. A. Pérez-Delgado, and J. F. Fitzsimons, "Fast graph operations in quantum computation," *Physical Review A*, vol. 93, no. 3, p. 032314, 2016.